LAPORAN TUGAS
PERTEMUAN 4

Studi Kasus: Aplikasi Pemesanan Kafe Kucing
Pengelolaan Props dan State pada React Native

Mata Kuliah	Aplikasi Berbasis Platform (BB244)
Nama Mahasiswa	I GUSTI BAGUS RYANDRA JAYA WIGUNA
NIM	240040042
Program Studi	Teknologi Informasi
Dosen Pengampu	MADE AGUS PUTRA SUBALI, S.Kom., M.Kom

1. Tautan Repositori Proyek (GitHub)

https://github.com/Gusryandra/Tugas-Pertemuan-4-Kafe-Kucing

2. Tampilan Aplikasi (Screenshot)

Berikut tampilan aplikasi yang dijalankan di Expo Snack (platform Web) pada tiga kondisi.



Gambar 1. Kondisi awal: total 0, semua qty 0, promo tidak tampil



Gambar 2. Total 3 (Kopi 1, Matcha 1, Croissant 1), promo belum tampil



Gambar 3. Total 5: label Diskon 10% muncul

Pada Gambar 3, total pesanan mencapai 5 sehingga kondisi totalCount >= 5 bernilai true dan label promo muncul secara dinamis, sesuai spesifikasi.

3. Source Code (App.js)

Seluruh komponen (App, OrderSummary, MenuItem) berada dalam satu berkas App.js. Emoji pada label promo ditulis sebagai escape Unicode (\u{1F389}) agar tampil benar di PDF; pada berkas App.js asli, emoji ditulis langsung.

```
import React, { useState } from 'react';
import {
  SafeAreaView,
  View,
  Text,
  TouchableOpacity,
  FlatList,
  StyleSheet,
} from 'react-native';

const DAFTAR_MENU = [
  { id: '1', nama: 'Kopi Susu Catnip', harga: 20000 },
  { id: '2', nama: 'Matcha Paw Latte', harga: 25000 },
  { id: '3', nama: 'Croissant Ikan Tuna', harga: 18000 },
];

const formatRupiah = (angka) =>
  'Rp ' + String(angka).replace(/\B(?=(\d{3})+(?!\d))/g, '.');

// Child Component 1: menerima props totalCount
function OrderSummary({ totalCount }) {
  return (
    <View style={styles.summaryBox}>
      <Text style={styles.summaryText}>Total Item Dipesan: {totalCount}</Text>
      {totalCount >= 5 && (
        <Text style={styles.promoText}>
          \u{1F389} Selamat! Anda Mendapatkan Diskon 10%
```

```
        </Text>
      )}
    </View>
  );
}

// Child Component 2: props + state mandiri (qty)
function MenuItem({ nama, harga, onTambahPesanan }) {
  const [qty, setQty] = useState(0);

  const handleTambah = () => {
    setQty((prevQty) => prevQty + 1); // update state lokal
    onTambahPesanan();                // beri tahu parent
  };

  return (
    <View style={styles.card}>
      <View style={styles.cardInfo}>
        <Text style={styles.menuName}>{nama}</Text>
        <Text style={styles.menuPrice}>{formatRupiah(harga)}</Text>
        <Text style={styles.menuQty}>Dipesan: {qty}</Text>
      </View>
      <TouchableOpacity style={styles.button} onPress={handleTambah}>
        <Text style={styles.buttonText}>Tambah Pesanan</Text>
      </TouchableOpacity>
    </View>
  );
}

// Parent Component: state global totalPesanan
export default function App() {
  const [totalPesanan, setTotalPesanan] = useState(0);

  const handleTambahPesanan = () => {
    setTotalPesanan((prevTotal) => prevTotal + 1);
  };

  return (
    <SafeAreaView style={styles.container}>
      <Text style={styles.header}>Kafe Kucing</Text>
      <OrderSummary totalCount={totalPesanan} />
      <FlatList
        data={DAFTAR_MENU}
        keyExtractor={(item) => item.id}
        contentContainerStyle={styles.list}
        renderItem={({ item }) => (
          <MenuItem
            nama={item.nama}
            harga={item.harga}
            onTambahPesanan={handleTambahPesanan}
          />
        )}
      />
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#FFF8F0', paddingTop: 40 },
  header: {
    fontSize: 26,
    fontWeight: 'bold',
    textAlign: 'center',
    color: '#5D4037',
    marginBottom: 12,
  },
  summaryBox: {
    backgroundColor: '#FFE0B2',
    marginHorizontal: 16,
    padding: 14,
    borderRadius: 12,
    marginBottom: 12,
  },
  summaryText: { fontSize: 16, fontWeight: '600', color: '#4E342E' },
```

```
  promoText: { marginTop: 6, fontSize: 14, fontWeight: 'bold', color: '#2E7D32' },
  list: { paddingHorizontal: 16, paddingBottom: 24 },
  card: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    backgroundColor: '#FFFFFF',
    padding: 14,
    borderRadius: 12,
    marginBottom: 12,
    elevation: 2,
  },
  cardInfo: { flex: 1, paddingRight: 8 },
  menuName: { fontSize: 16, fontWeight: 'bold', color: '#3E2723' },
  menuPrice: { fontSize: 14, color: '#6D4C41', marginTop: 2 },
  menuQty: { fontSize: 13, color: '#8D6E63', marginTop: 4 },
  button: {
    backgroundColor: '#8D6E63',
    paddingVertical: 10,
    paddingHorizontal: 12,
    borderRadius: 8,
  },
  buttonText: { color: '#FFFFFF', fontWeight: '600' },
});
```

4. Ringkasan Arsitektur Komponen

Komponen	Peran	Props diterima	State
App (Parent)	Menyimpan DAFTAR_MENU, merender header, OrderSummary, dan daftar MenuItem	-	totalPesanan (number, awal 0)
OrderSummary	Menampilkan "Total Item Dipesan: X" dan label diskon bila totalCount >= 5	totalCount (number)	-
MenuItem	Kartu satu menu dan tombol "Tambah Pesanan"	nama (string), harga (number), onTambahPesanan (function)	qty (number, awal 0)

Aturan yang dipenuhi: useState digunakan di App dan MenuItem; props tidak pernah dimodifikasi langsung di child (hanya dibaca atau dipanggil); komponen inti View, Text, TouchableOpacity, FlatList, dan StyleSheet; props diterima dengan object destructuring.

5. Jawaban Analisis

Soal Analisis A: Mengapa qty disimpan di MenuItem, sedangkan totalPesanan disimpan di App?

Letak sebuah state ditentukan oleh siapa saja yang membutuhkan data tersebut. **qty** hanya relevan bagi satu kartu menu: tidak ada komponen lain yang perlu mengetahui berapa porsi Kopi Susu Catnip yang dipesan. Dengan menyimpannya di MenuItem, setiap kartu memiliki salinan state sendiri yang independen (menekan tombol pada Matcha Paw Latte tidak mengubah qty menu lain), dan komponen tetap mandiri serta mudah dipakai ulang.

Sebaliknya, **totalPesanan** dibutuhkan oleh komponen lain, yaitu OrderSummary, yang bukan induk dari MenuItem. Di React, data mengalir satu arah (top-down) dari parent ke child melalui props, dan antar-saudara (sibling) tidak dapat saling membaca state. Maka state dinaikkan ke ancestor terdekat yang sama, yaitu App (konsep lifting state up). App

menyimpan totalPesanan, meneruskannya ke OrderSummary sebagai props totalCount, dan memberi MenuItem fungsi callback onTambahPesanan agar child dapat meminta parent memperbarui state tersebut tanpa memutasi props. Dengan demikian terdapat satu sumber kebenaran (single source of truth) untuk total pesanan.

Soal Analisis B: Apa yang terjadi pada tampilan ketika tombol "Tambah Pesanan" ditekan? Jelaskan alur re-render yang dipicu oleh setter state!

Alur ketika tombol ditekan:

1. Event onPress memanggil handleTambah di MenuItem yang ditekan.
2. setQty(prev => prev + 1) dijalankan. React tidak mengubah nilai qty secara langsung, melainkan menjadwalkan pembaruan dan menandai komponen MenuItem tersebut perlu dirender ulang.
3. onTambahPesanan() (props callback) dipanggil. Fungsi ini sebenarnya handleTambahPesanan milik App, yang menjalankan setTotalPesanan(prev => prev + 1) sehingga App juga ditandai perlu dirender ulang.
4. Kedua pembaruan berada dalam satu event handler, sehingga React (batching otomatis) menggabungkannya dan melakukan satu kali proses render.
5. Fungsi App dieksekusi ulang dengan totalPesanan baru. Seluruh child yang dirender App ikut dieksekusi ulang: OrderSummary menerima totalCount baru, dan setiap MenuItem dipanggil ulang dengan props yang sama. MenuItem yang ditekan membaca qty terbaru dari state-nya, sedangkan MenuItem lain tetap memakai qty masing-masing karena state tiap instance dipertahankan oleh React.
6. React membandingkan hasil render baru dengan sebelumnya (reconciliation), lalu hanya menerapkan perbedaan ke tampilan native: teks "Dipesan: qty" pada kartu yang ditekan dan teks "Total Item Dipesan: X" berubah. Ketika totalCount mencapai 5, kondisi totalCount >= 5 bernilai true sehingga label "Selamat! Anda Mendapatkan Diskon 10%" muncul secara dinamis.

Ringkasnya, setter state memicu render ulang komponen pemilik state beserta turunannya, dan tampilan selalu merupakan hasil fungsi dari state dan props terbaru.

6. Skenario Pengujian

Aksi	Hasil yang diharapkan
Aplikasi dibuka	Total Item Dipesan: 0; semua "Dipesan: 0"; label promo tidak tampil
Tekan "Tambah Pesanan" pada Kopi Susu Catnip 2x	Dipesan: 2 pada kartu tersebut; Total Item Dipesan: 2
Tekan Matcha Paw Latte 2x dan Croissant Ikan Tuna 1x	Masing-masing qty 2 dan 1; Total Item Dipesan: 5; label diskon 10% muncul